

Project Report

AI 기반 인프라 안전 점검 (Crack 자동 분류 모델)

팀 : B

프로젝트 목표

- 1) 콘크리트 표면 이미지를 분석하여 **크랙 유무 및 크랙 유형(hairline / wide)**을 자동 분류하는 **AI 기반 이미지 분류 모델을 개발**하고,
- 2) 이를 웹 기반 UI(Streamlit)를 통해 누구나 손쉽게 사용할 수 있도록 구현한다.

슬라이드 구성

01.

데이터셋 구성

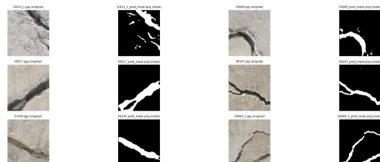
Kaggle
- Concrete Crack Images
for Classification



02.

데이터 전처리

Positive Images Segmentation



03.

데이터 폭 기반 Class binary 분류

Positive Images 분류
Hairline vs wide



04.

MobileNetv2 기반 3-class 분류

Hairline vs wide vs negative
이미지 분류

05.

MobileNetv2 기반 3-class 분류 (Oversampling)

Oversampling을 통한
분류 모델 성능 개선

06.

Streamlit 기반 Crack 분류 웹 구현



사용 데이터 셋 구성



Negative image

Positive image

1. 데이터 출처 : Kaggle –Concrete Crack Images for Classification

- 총 이미지 수 : 40,000장
 - Positive (Crack 있음) : 20,000장
 - Negative (Crack 없음) : 20,000장
- 이미지 해상도 : 227 x 227
- 각 이미지에는 단일 객체(균열 여부)가 포함되어 있음
- 초기 분류 기준 : Crack 유무에 따른 Binary Classification

<https://www.kaggle.com/datasets/arnavr10880/concrete-crack-images-for-classification>

Crack Images Segmentation 기반 마스크 생성

- Positive 이미지 중 271장을 수작업으로 Segmentation
 - 도구 **LabelMe** (Ai Polygon 사용)
 - 저장 형식 : **.json 파일** (LabelMe 포맷)
- 수작업 Segmentation 기반으로 Segmentation 모델 학습 (**U-Net** 모델 사용)
- 학습된 모델을 사용하여 20,000장의 Positive 이미지 Segmentation 작업 진행
- Segmentation 결과는 이후 **crack의 폭** 기반 분류에 활용

💡 U-Net 사용 이유

- 픽셀 단위 정확도 ↑
- 구조가 간단하고 학습 효율 높음
- 소량 데이터로도 일반화 가능

💡 왜 수작업 271장만?

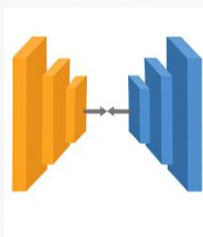
- 전수 라벨링은 시간 부담 큼
- 일부 대표 이미지로 학습
→ 나머지는 전이학습 기반 자동 segmentation

요약

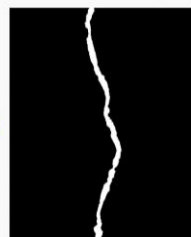
데이터 전처리



LabelMe
수작업

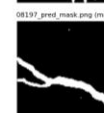
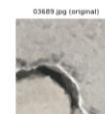
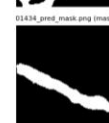
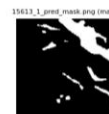
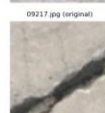


U-Net 학습



전체 이미지
자동 segmentation

Segmentation masking 결과 Images



Positive Image를 Hairline vs Wide으로 분류

- Segmentation 마스크를 기반으로 크랙의 폭(두께)을 정량적으로 분석
- 크랙의 최대 또는 평균 폭을 측정하여 폭 기준 이진 분류 수행
 - 폭 < 임계 값 (4.5 pixel) -> **Hairline Crack**
 - 폭 > 임계 값 (4.5 pixel) -> **Wide Crack**
- 공존할 경우 wide crack 우선 분류
- 분류된 결과는 이후 3-Class 분류 모델의 라벨로 사용됨

💡 왜 크랙 폭을 기준으로 분류했는가?

- LabelMe로 모든 이미지를 직접 라벨링 하기에는 시간적 제약이 크기 때문에,
- segmentation을 통해 얻은 마스크를 활용하여 물리적 특징(폭) 기반으로 자동 분류

요약

Classification of Crack Type

Quantitative analysis of crack width based on threshold

$$\text{avg_width} \leq 4.5$$

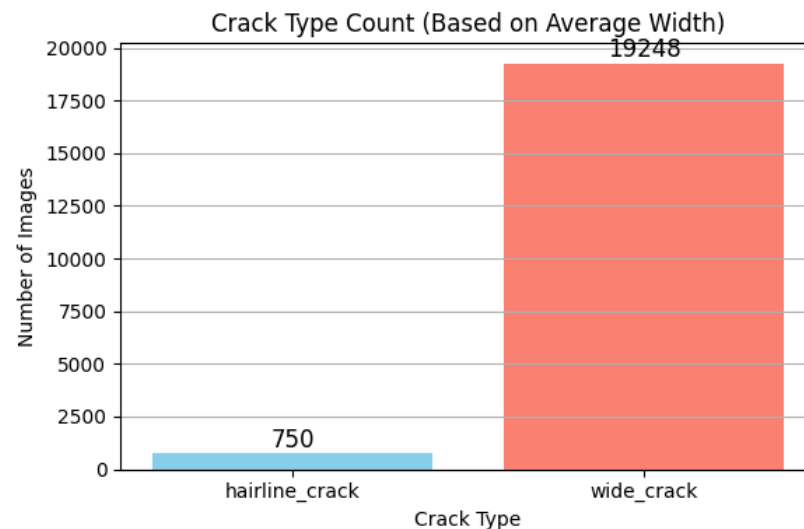
헤어라인 균열
Hairline Crack



폭이넓은 균열
Wide Crack



Positive Images Binary 분류 결과



3-Class 분류 모델 구성 (MobileNetV2)

Crack 이미지 3-Class 분류 모델 구축

분류 대상 클래스	1. hairline_crack
	2. wide_crack
	3. negative (crack 없음)
입력 데이터	- positive 20,000장 -> segmentation + 폭 분류로 hairline/wide 레이블 지정 - negative 20,000장 포함하여 총 3 class 구성
모델 아키텍처	MobileNetV2
전이학습 방식	Feature Extractor로 사용 후 일부 레이어 Fine-tuning
입력 이미지 크기	128 x 128
출력층	softmax (3개 클래스)

💡 왜 MobileNetV2를 사용했는가?

- 파라미터 수가 적고 계산 효율이 뛰어나 학습 속도가 빠름
- 사전 학습된 가중치(ImageNet 기반)를 활용한 전이학습 가능
- 소량의 데이터로도 높은 분류 성능을 낼 수 있음

💡 일부 레이어만 Fine-tuning한 이유?

- 전체 레이어를 학습시키면 오히려 과적합 위험이 커짐
- 상위 레이어 일부만 학습함으로써 crack 이미지에 특화된 특징 학습 가능
- 학습 시간 단축 + 모델의 일반화 성능 유지

학습 정보

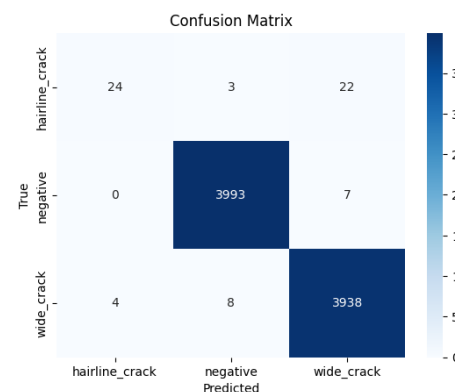
Loss Function	Categorical Crossentropy
Optimizer	Adam (lr=1e-4)
Augmentation	수평 뒤집기, 줌, 밝기 조절
Validation split	0.2

Confusion Matrix

Class	Precision	F1-score
hairline_crack	0.86	0.62
wide_crack	0.99	0.99
negative	1	1

Macro accuracy : 0.9945

Macro F1_score : 0.8270



* 문제점 : hairline_crack의 image 불균형으로 f1-score 낮음 (0.62)

불균형 클래스 개선을 위한 Oversampling 및 재 학습

- 초기 학습 결과 : hairline_crack 클래스의 F1-score가 낮게 나타남 (0.62)
- 원인 분석:
 - Hairline_crack 이미지 수가 wide_crack, negative에 비해 매우 적음
 - 모델이 hairline 패턴을 충분히 학습하지 못함
- 해결 전략:
 - hairline_crack 이미지를 oversampling하여 학습 데이터에서 클래스 균형 조절

💡 Oversampling 구현 방식

- wide_crack 클래스 수 기준으로 hairline 이미지 복제
- copy_000_ 형태로 파일명 변경하여 디렉토리 구성

💡 MobileNetV2 $\alpha=0.35$ 지정 이유

- 학습 속도 및 자원 소모 문제 해결
- 채널 수 축소 $\rightarrow$ 모델 경량화 $\rightarrow$ 연산량 감소
- 성능 손실 없이 학습 시간 단축 기대

Oversampling 코드 내용 요약

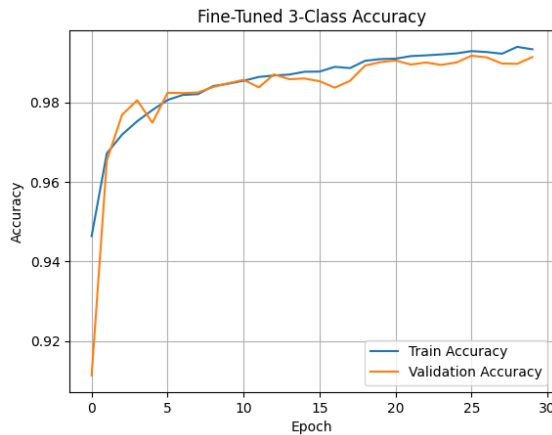
- 기존 클래스 구조: negative, hairline_crack, wide_crack
- hairline_crack 클래스만 복제하여 oversampling한 디렉토리 구성:

```
oversampled_dir = '/content/3class_dataset_oversampled'
```

- 복제 방식:
 - hairline_crack 이미지 수가 적으므로 부족한 수만큼 원본 hairline 이미지를 무작위로 복제(copy)
(예: copy_0_00023.jpg 식으로 이름 바꿈)
 - 이후 모델 학습 시, oversampled 데이터를 불러와 학습
- MobileNetV2의 일부 레이어만 Fine-tuning 하여 재학습
 - alpha 값 0.35 지정

불균형 클래스 개선을 위한 Oversampling 및 재 학습

학습 결과

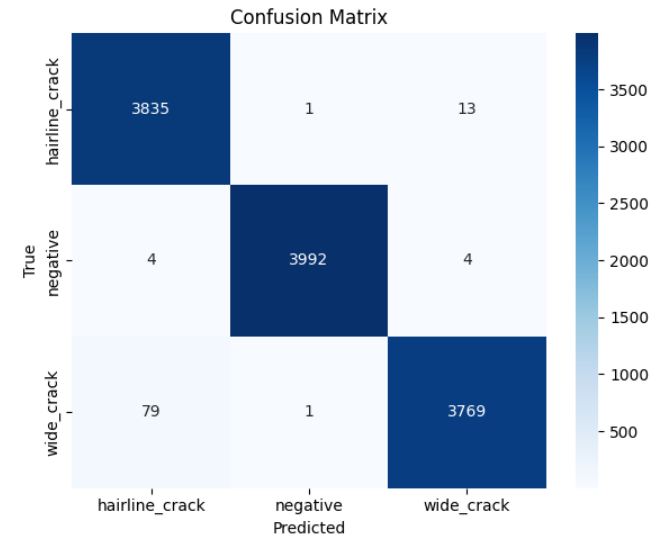


Confusion Matrix

Class	Precision	F1-score
hairline_crack	0.98	0.99
wide_crack	0.99	0.99
negative	1	1

Macro accuracy : 0.9913

Macro F1_score : 0.9912



성능 비교 요약

항목	Oversampling 전	Oversampling 후
Accuracy	99.45%	99.13%
F1 (macro avg)	0.827	0.9912
<u>F1 (hairline)</u>	<u>0.62</u>	<u>0.99</u>
클래스 imbalance	있음	해소됨

💡 성능 개선의 핵심 요인

- **Oversampling**으로 hairline_crack 데이터 비중 ↑
- **Data Augmentation**을 병행하여 과적합 방지
- MobileNetV2의 일부 레이어 Fine-tuning 적용

이미지 업로드 기반 크랙 분류 웹 구축

- 사용 도구 : **Streamlit** + TensorFlow + PIL + JSON
- 사용자가 웹 브라우저에서 Crack 이미지 업로드
- 업로드된 이미지를 128 x 128로 전처리한 뒤, 사전 학습된 MobileNetV2 기반 모델로 분류 수행
- 결과는 예측된 crack 유형(hairline / wide / negative)과 신뢰도(confidence)로 표시

💡 Streamlit 활용 이유

- 빠르고 간단하게 머신러닝 모델을 웹 UI로 배포 가능
- 코드 50줄 이내로 실제 서비스 가능한 프로토타입 완성
- 개발자가 아닌 사용자도 이미지 업로드만으로 분류 가능

실제 웹 상 사용 이미지

Crack Type Classifier

업로드한 이미지에 대해 크랙 유무 및 유형을 분류합니다.

이미지를 업로드하세요 (JPG, PNG)

Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

Browse files

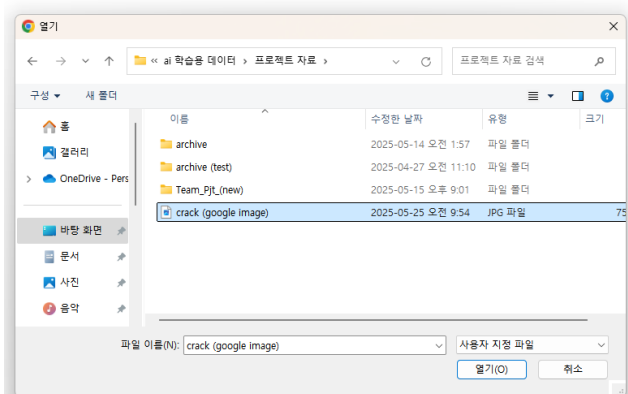
Crack Type Classifier

업로드한 이미지에 대해 크랙 유무 및 유형을 분류합니다.

이미지를 업로드하세요 (JPG, PNG)

Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

Browse files



Crack Type Classifier

업로드한 이미지에 대해 크랙 유무 및 유형을 분류합니다.

이미지를 업로드하세요 (JPG, PNG)

Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

Browse files

crack (google image).jpg 74.0KB

The use_column_width parameter has been deprecated and will be removed in a future release. Please utilize the use_container_width parameter instead.



업로드된 이미지

✅ 예측 결과: **wide_crack**

신뢰도: 53.91%

홈페이지 (이미지 업로드 전)

이미지 업로드 중

크랙 이미지 예측 완료